

Zephyr RTOS

Hardware Abstraction

Basics

K P Lakshmeesh (22B1246)
EE BTech, IIT Bombay

Note: This documentation is based on Zephyr RTOS version 4.2.0.

Importance of portability:

- Typically, beginners at any embedded system hobby project make a software application which is highly coupled with hardware; where pin assignments, registers, driver configurations are hardcoded into the codes.
- However, this practice is not at all reliable for large scale projects or industry applications. When we are forced to try out other microcontrollers due to supply chain constraints, cost optimisations, or for upgrading systems, engineers would be forced to rewrite huge chunks of codes manually.
- Thus, we need something which can separate the software logic from the hardware description. Such that we reuse the same codes for any microcontroller we wish to use, with minimal code changes.
- Zephyr RTOS is famous for offering portability across a wide range of microcontrollers, with an active community maintaining it.

Some foundational concepts:

- **Device trees (.dts/.dtsi):**
 - o It is a tree-like structured file, which describes the hardware layout of a microcontroller
- **Device drivers:**
 - o Low level C codes which contain the register manipulations required to talk with the hardware units.
- **Binding files (.yaml):**
 - o Validation rulebooks used during compilation of the hardware description.
 - o They specify the properties which are mandatorily required, what kind of data are allowed within a node, naming conventions for device trees overlays etc.
- **Overlay files (.overlay):**
 - o These are application specific files, which can be used to modify the pre-existing device tree properties without altering the core files provided by the vendor.
 - o We may want to use separate overlay files for separate applications, overriding the same default device tree.
- **Project configuration (prj.conf):**
 - o It deals with the software architecture. It is a configuration file used to select which specific modules, kernel features, or hardwares are active in the application.
 - o Eg:
we may want to pull in libraries to be able to run TFLM engines for an ML application,
so we set `CONFIG_TENSORFLOW_LITE_MICRO=y`
or we may want to include software layers involving ADC, so set `CONFIG_ADC=y`.
We may also dictate other properties like the stack/heap size, clock frequency, thread scheduling algorithm etc. using it.
 - o It is helpful because microcontrollers are generally resource constrained, and we wish to pull in only the essential software features to keep the final RAM footprint minimal.
- **Zephyr snippets:**
 - o A snippet is a reusable configuration bundle designed to avoid repetitive copy-pasting of configs (prj.conf) across applications. We can specify certain snippets during build to avail standard features.
 - o Eg:
by default, pico device tree routes the console output to uart pins. But, to capture data via USB on a PC, we can just include the “cdc-acm-console” snippet instead of having to buy serial-to-USB probes. It sets up a virtual COM port, and re-routes the console logging from uart to the virtual usb port.
(note: setting `CONFIG_USB_DEVICE_INITIALIZE_AT_BOOT=y`; in prj.conf is also required for this example)

How to write an overlay file?

We check these files to configure devices properly:

- **Device tree files (.dts/.dtsi) (soc + board):**
 - It defines all the microcontroller peripherals (ADC, DMA, SPI etc.)
 - Eg:
rp2040.dtsi (chip specific; `zephyr/dts/arm/raspberrypi/rpi_pico`),
rpi_pico-common.dtsi (board specific; `zephyr/boards/raspberrypi/rpi_pico`)
(there's few more mentioned in the `#include` headers of the board files)
 - Most peripherals have `status="disabled"` in chip specific file. However, in the board specific file, several of these (eg: adc) have been set to `status="okay"`.
Some (like DMA) are not mentioned here, which means they are still disabled.
If we wish to disable ADC or enable DMA, or modify some configuration, we have to manually override them through an overlay file.
 - Let's see rp2040.dtsi in more detail, about ADC in specific:
(you may also refer: `zephyr/dts/bindings/base/base.yaml`)
the most relevant for making our overlay files are 'compatible', 'status'. We generally don't worry about others. They are also simply explained below.

```
zephyrproject > zephyr > dts > arm > raspberrypi > rpi_pico > rp2040.dtsi
26 / {
189     soc {
308         adc: adc@4004c000 {
309             compatible = "raspberrypi,pico-adc";
310             reg = <0x4004c000 DT_SIZE_K(4)>;
311             resets = <&reset RPI_PICO_RESETS_RESET_ADC>;
312             clocks = <&clocks RPI_PICO_CLKID_CLK_ADC>;
313             interrupts = <22 RPI_PICO_DEFAULT_IRQ_PRIORITY>;
314             interrupt-names = "adc0";
315             status = "disabled";
316             #io-channel-cells = <1>;
317         };

```

- The node's "phandle label" is 'adc' (name before colons used as reference at other places)
- "Compatible" string is a unique key used to match the node against both its validation rulebook (.yaml) and its corresponding driver file (`adc_rpi_pico.c`).
The Pico ADC driver (`zephyr/drivers/adc/adc_rpi_pico.c`) defines `"#define DT_DRV_COMPAT raspberrypi_pico_adc"`, which is used by the compiler while matching the compile target (commas and hyphens are converted to underscores by Zephyr). You can see `DT_DRV_COMPAT` macro being utilised in `devicetree.h` (`zephyr/include/zephyr/devicetree.h`).
In the driver code, several functions are defined for the working of the Pico ADC, and they are tied to Zephyr's standard APIs via the macros declared towards the end of file.
- "status": if the device is enabled/disabled
- Hardware registers (reg, clock, interrupts, resets): hardwired specifications that map memory regions, clocks, and interrupt lines. These lines don't need to be altered inside your application overlay.
 - "reg" tells the base address of the ADC hardware registers (obtained from rp2040 datasheet). 4KB memory is allocated for these registers.
 - "resets", "clocks" tie the correct reset and clock signals to the ADC system.
 - "interrupt" wires to the 22 ISR line with default priority. "interrupt-names" is for low-level code convenience.
- "#io-channel-cells=<1>": the number of additional cell arguments (here 1) to be given alongside its handle pointer to resolve the channel index (eg: `<&adc 0>`)

- **YAML binding files (.yaml):**

- .yaml files act as the validation rulebook during the build phase to confirm your overlays use permitted properties and structures.

Refer to:

“adc-controller.yaml”(common for all adc) , “raspberrypi,pico-adc.yaml” (pico board specific)

- Eg: Adc-controller.yaml mandates that individual adc channels be declared as child nodes, and enforces a naming standard starting with “channel” (eg:channel@0)

```
/ {
  zephyr,user {
    io-channels = <&adc 0>;
  };
};

&adc {
  #address-cells = <1>;
  #size-cells = <0>;

  channel@0 {
    reg = <0>;
    zephyr,gain = "ADC_GAIN_1";
    zephyr,reference = "ADC_REF_INTERNAL";
    zephyr,acquisition-time = <ADC_ACQ_TIME_DEFAULT>;
    zephyr,resolution = <12>;
  };
};
```

(zephyr/samples/drivers/adc/adc_dt/boards/rpi_pico.overlay)

- We refer to the ‘adc’ phandle (as we had seen before) using ‘&adc’.
- As required, we have named the node corresponding adc0 as “channel@0”. (what you write after “channel” is based on your convenience)
- “reg” tells the channel index. For eg, here 0 corresponds to ADC_CH0 / GPIO26 on pico.
- Many properties specific to the working of the ADC are also declared (gain, reference voltage, acquisition time, resolution). The validity of these values is checked by the adc_rpi_pico.c driver file.
All required properties and their possible values can be seen within the adc-controller.yaml file.
- You may have observed that “#address-cells” and “#size-cells” were also mentioned in the board .yaml file. However, they are again declared in the overlay. It’s because, as mentioned before, .yaml is only used for validation of properties during building. It is required here for immediate parsing guidance.
- #io-channel-cells is already specified in rp2040.dtsi, thus its not written here again.
- We also see the zephyr,user block in the image. This is an abstract routing mechanism, which ties the generic application level code to specific physical hardware nodes. There are also other ways to do this instead of “zephyr,user”. (mentioned next)

Abstract routing mechanisms:

- These help during compile time mapping between the fully generic application code and the specific hardware details of the chosen microcontroller. In device tree we provide information regarding hardware via nodes. Then in the application code, we use standard zephyr macros to access these devices.
- During ‘west build’, all these abstract device tree nodes are translated into hardcoded memory-mapped registers to access the devices.
- 3 methods are available in zephyr:
- **“zephyr,user” node:**
 - (refer this for better understanding: [\[2\]](#))
 - It is a virtual software node (not an actual hardware node). It can be used to more quickly access the hardware nodes from the application through certain APIs.
eg: Bundling ADC arrays to iterate through them via loops in application code (see samples/drivers/adc/adc_dt for example). we can use DT_FOREACH_PROP_ELEM(DT_PATH(zephyr_user),

io_channels, DT_SPEC_AND_COMMA) to easily access all the ADC nodes specified within `zephyr,user {io-channel = <&adc 0>, <&adc 1>, <&adc 2>;}`;

Otherwise, we would have to use `DT_NODELABEL()` and `DT_CHILD()` macros for each adc channel one by one.

- Naming format is strictly “zephyr,user”
- While implementing for ADC, this API reference [\[3\]](#) helps with syntax and various APIs to use the io-channels data.

○ **“chosen” node:**

(reference: [\[4\]](#), [\[5\]](#))

- It is used to dictate a set of hardware defaults.
- Naming format is fully custom. You can name a custom property as “mic-adc” as well. Zephyr follows a “zephyr,<text>” convention for its properties.

```
16 / {
17     chosen {
18         zephyr,sram = &sram0;
19         zephyr,flash = &flash0;
20         zephyr,flash-controller = &ssi;
21         zephyr,console = &uart0;
22         zephyr,shell-uart = &uart0;
23         zephyr,code-partition = &code_partition;
24     };
25
26     aliases {
27         rtc = &rtc;
28         watchdog0 = &wdt0;
29     };
30
31     pico_header: connector {
32         compatible = "raspberrypi,pico-header";
33         #gpio-cells = <2>;
34         gpio-map-mask = <0xffffffff 0xffffffffc0>;
35         gpio-map-pass-thru = <0 0x3f>;
36         gpio-map = <0 0 &gpio0 0 0>, /* GP0 */
37                 <1 0 &gpio0 1 0>, /* GP1 */
```

(eg code snippet from `rpi_pico-common.dtsi`)

- It maps specific predefined tasks to a chosen hardware node.

For eg, we can tie ‘zephyr,console’ with usb or uart depending on our need, for logging console outputs.

- (&uart0 was the default. I had changed it to &usbd since I wasn’t using uart lines for console output. However, if we use the usb-cdc-acm zephyr snippet, it will overwrite from uart default to usb; its not wise to touch the device tree in general)
- also see “uart_console.c” where this chosen zephyr,console node is called
- For an ADC, we could also create a custom “zephyr,microphone-adc” property to set a system default for an analog speech source.

○ **“aliases” node:**

(reference: [\[4\]](#), [\[6\]](#))

- Useful for directly pointing at some deeply nested hardware node. We can assign human-readable short aliases (eg: rtc, watchdog0).
- Naming format is fully custom.
- Instead of referring long-complex node paths (like “/soc/adc@4004c000/channel@0”), we can directly assign an alias (like mic = &adc0, where we put ‘adc0’ label to channel@0 node). Then we can access it using `ADC_DT_SPEC_GET(DT_ALIAS(mic))` macro in the application code.